

## Actividad 01 — Limpieza y Exploración de Datos

Comparación: Enfoque Manual vs Desarrollo Asistido con Claude Code

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)

Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-10-05

## Descargas y Enlaces

---

# Introducción: La Actividad de Limpieza de Datos

**Contexto:** Recibes un archivo Excel con estadísticas de mujeres en ajedrez de federaciones mundiales.

La actividad tradicional (Actividad 01 original) requiere:

1. Cargar datos desde Excel (hoja “data”)
2. Cargar diccionario de variables (hoja “variables”)
3. Crear un diccionario de variables completo
4. Realizar estadística descriptiva
5. Limpiar y estandarizar variables
6. Crear nuevas variables si es necesario
7. Entregar un reporte con explicaciones y código

**Pregunta pedagógica:** ¿Cómo acelerar este flujo sin sacrificar calidad, usando IA?

Este documento muestra la comparación directa: **enfoque manual vs asistido con Claude Code**.

---

# Enfoque 1: Manual (Tradicional)

## Paso 1: Cargar librerías y datos

Sin IA, escribirías el código manualmente. El profesor te entrega un archivo Excel con dos hojas: - “**data**”: los datos de mujeres en ajedrez - “**variables**”: diccionario que explica cada columna

```
# Cargar librerías
library(readxl)
library(tidyverse)
library(skimr)

# Cargar datos
data <- read_excel("[archivo Excel entregado]", sheet = "data")
variables_dict <- read_excel("[archivo Excel entregado]", sheet = "variables")

# Ver estructura
head(data)
str(data)
```

**Tiempo invertido:** ~5 minutos (recordar sintaxis, buscar ayuda si olvidas argumentos).

---

## Paso 2: Exploración Manual

```
# Diccionario manual
print(variables_dict)

# Estadística descriptiva manual
summary(data)
skim(data)

# Buscar valores faltantes
data %>%
  summarise(across(everything(), ~sum(is.na(.))))

# Visualizar distribuciones
data %>%
```

```

pivot_longer(everything()) %>%
ggplot(aes(x = value)) +
geom_histogram() +
facet_wrap(~name, scales = "free")

```

**Tiempo invertido:** ~15 minutos (prueba y error, ajustes visuales).

---

## Paso 3: Limpieza y Transformación Manual

```

# Estandarizar nombres
data_clean <- data %>%
  rename_all(tolower) %>%
  rename_all(~gsub(" ", "_", .))

# Detectar y manejar valores atípicos
data_clean <- data_clean %>%
  mutate(
    ranking = as.numeric(ranking),
    age = as.numeric(age),
    rating = as.numeric(rating)
  )

# Crear nuevas variables
data_clean <- data_clean %>%
  mutate(
    rating_category = case_when(
      rating >= 2300 ~ "Grandmaster",
      rating >= 2200 ~ "Master",
      rating >= 2000 ~ "Expert",
      TRUE ~ "Other"
    ),
    age_group = cut(age, breaks = c(0, 20, 30, 40, 50, 100),
      labels = c("<20", "20-30", "30-40", "40-50", "50+"))
  )

```

**Tiempo invertido:** ~20 minutos (ajustes iterativos, decisiones ad-hoc).

---

## Paso 4: Generar Reporte Manual

```

# Crear resumen
resumen <- data_clean %>%
  group_by(rating_category, age_group) %>%
  summarise(

```

```
n = n(),
mean_rating = mean(rating, na.rm = TRUE),
sd_rating = sd(rating, na.rm = TRUE)
)

# Guardar en archivo
write.csv(resumen, "resumen_limpieza.csv")
```

**Tiempo total aproximado: 50-60 minutos** (incluyendo debugging, consultas, retoques).

---

# Enfoque 2: Asistido con Claude Code

## Estrategia: Describe qué necesitas en lenguaje natural

En lugar de escribir código línea por línea, describes la tarea a Claude Code:

### Prompt al asistente:

“Necesito limpiar un dataset de ajedrez desde Excel. - Carga dos hojas: ‘data’ y ‘variables’  
- Crea un reporte que incluya: 1. Diccionario de variables (nombre, tipo, descripción) 2. Estadística descriptiva completa 3. Detección de valores faltantes y atípicos 4. Estandarización de nombres de columnas 5. Nuevas variables: categoría de rating (GM, Master, Expert, Otros) y grupos de edad 6. Visualizaciones de distribuciones - Exporta todo a un archivo de reporte HTML”

---

## Lo que Claude Code genera automáticamente

Claude Code crearía un script completo optimizado:

```
library(readxl)
library(tidyverse)
library(skimr)

# 1. CARGAR DATOS
data <- read_excel("[archivo Excel entregado]", sheet = "data")
variables_dict <- read_excel("[archivo Excel entregado]", sheet = "variables")

# 2. DICCIONARIO DE VARIABLES
var_dict_formatted <- variables_dict %>%
  rename(
    "Variable" = starts_with("nombre") | starts_with("Nombre"),
    "Tipo" = starts_with("tipo") | starts_with("Tipo"),
    "Descripción" = starts_with("descr") | starts_with("Descr")
  )

# 3. ESTADÍSTICA DESCRIPTIVA
skim_stats <- skim(data)
summary_stats <- data %>%
```

```

summarise(across(where(is.numeric),
  list(
    n = ~n(),
    mean = ~mean(., na.rm = TRUE),
    sd = ~sd(., na.rm = TRUE),
    min = ~min(., na.rm = TRUE),
    q25 = ~quantile(., 0.25, na.rm = TRUE),
    median = ~median(., na.rm = TRUE),
    q75 = ~quantile(., 0.75, na.rm = TRUE),
    max = ~max(., na.rm = TRUE),
    missing = ~sum(is.na(.))
  ),
  .names = "{.col}_{.fn}"
))

# 4. VALORES FALTANTES
missing_report <- data %>%
  summarise(across(everything(),
    ~paste(sum(is.na(.)), "de", n(), "valores faltantes")))

# 5. LIMPIEZA Y ESTANDARIZACIÓN
data_clean <- data %>%
  rename_all(tolower) %>%
  rename_all(~gsub(" ", "_", .)) %>%
  mutate(across(where(is.character), str_trim))

# 6. NUEVAS VARIABLES
data_clean <- data_clean %>%
  mutate(
    rating_numeric = as.numeric(rating),
    age_numeric = as.numeric(age),
    rating_category = case_when(
      rating_numeric >= 2300 ~ "Grandmaster",
      rating_numeric >= 2200 ~ "Master",
      rating_numeric >= 2000 ~ "Expert",
      TRUE ~ "Otro"
    ),
    age_group = cut(age_numeric,
      breaks = c(0, 20, 30, 40, 50, 100),
      labels = c("<20", "20-30", "30-40", "40-50", "50+"))
  )

# 7. VISUALIZACIONES
plot_rating_dist <- data_clean %>%
  ggplot(aes(x = rating_numeric)) +
  geom_histogram(binwidth = 50, fill = "#FF0000", alpha = 0.7) +
  labs(title = "Distribución de Ratings",

```

```

    x = "Rating", y = "Frecuencia") +
  theme_minimal()

plot_age_dist <- data_clean %>%
  ggplot(aes(x = age_numeric)) +
  geom_histogram(binwidth = 5, fill = "#0066CC", alpha = 0.7) +
  labs(title = "Distribución de Edades",
        x = "Edad", y = "Frecuencia") +
  theme_minimal()

# 8. RESUMEN POR CATEGORÍAS
summary_by_category <- data_clean %>%
  group_by(rating_category, age_group) %>%
  summarise(
    n = n(),
    mean_rating = mean(rating_numeric, na.rm = TRUE),
    sd_rating = sd(rating_numeric, na.rm = TRUE),
    mean_age = mean(age_numeric, na.rm = TRUE),
    .groups = 'drop'
  )

# 9. EXPORTAR RESULTADOS
write.csv(summary_by_category, "resumen_limpieza.csv", row.names = FALSE)

```

---

## Ventajas del Enfoque Asistido

| Aspecto                 | Manual                                 | Con IA                                 |
|-------------------------|----------------------------------------|----------------------------------------|
| <b>Tiempo de código</b> | 50-60 min                              | 5-10 min                               |
| <b>Boilerplate</b>      | Completo (cargas, librerías)           | Generado automáticamente               |
| <b>Cobertura</b>        | A menudo incompleta<br>(olvidas casos) | Exhaustiva                             |
| <b>Documentación</b>    | Manual, lleva tiempo                   | Generada con explicaciones             |
| <b>Iteraciones</b>      | Reescribir cada cambio                 | Pedir cambios en lenguaje natural      |
| <b>Debugging</b>        | Lees mensajes de error solo            | Asistente explica y propone soluciones |

---

# Comparación en Tiempo Real

**Escenario:** “Quiero visualizar age\_group vs rating\_category”

**Manual (Sin IA)**

```
# Buscas en Google, Stack Overflow, recuerdas sintaxis de ggplot
# Pruebas varios geoms (bar, tile, violin)
# Ajustas colores, temas, etiquetas

data_clean %>%
  ggplot(aes(x = age_group, y = rating_category, fill = n())) +
  geom_tile() +
  # ... (10 minutos de ajustes)
```

**Tiempo:** ~15 minutos

**Con IA**

Prompt: "Dame un gráfico de tiles que muestre age\_group vs rating\_category, con conteos en color y etiquetas claras"

Claude Code genera el código optimizado **en segundos**. Solo ajustas si necesitas.

**Tiempo:** ~2 minutos (incluido ajuste)

---

# Actividad para Estudiantes: Tu Turno

## Instrucciones (Con IA)

### Opción A: Maximalista (Recomendada para aprender)

**Paso 1: Prepara tu workspace** - Abre Positron - Crea un proyecto nuevo - Ubica el archivo Excel entregado (con hojas “data” y “variables”)

### Paso 2: Usa Claude Code para exploración inicial

Abre Claude Code en Positron y pide (reemplaza `archivo.xlsx` con el nombre del archivo):

“Carga el archivo Excel (hojas ‘data’ y ‘variables’) y dame un resumen inicial: - Dimensiones del dataset - Tipos de datos - Valores faltantes por columna - Top 5 filas”

### Paso 3: Itera sobre la limpieza

“Ahora estandariza los nombres de columnas (tolower, sin espacios), convierte campos numéricos si es necesario, y crea estas variables: - `rating_elite`: TRUE si `rating`  $\geq$  2200 - `years_active`: diferencia entre max y min de año”

### Paso 4: Visualiza resultados

“Crea 3 gráficos: 1. Distribución de ratings (histogram) 2. Rating por país (top 10) 3. Correlación entre edad y rating”

### Paso 5: Genera reporte

“Exporta a un archivo HTML con: - Tabla de diccionario de variables - Resumen estadístico por país - Los 3 gráficos anteriores”

---

## Opción B: Enfoque Híbrido (Equilibrado)

Si prefieres un balance entre escribir código y usar IA:

1. Carga datos manualmente en R
  2. Usa Claude Code para la estandarización compleja
  3. Visualiza usando ggplot2 (escribes el esquema básico, IA refina)
  4. Genera reporte en Quarto (mezcla de código propio + IA)
-

## Opción C: Minimalista (Enfoque “Prompt-Driven”)

Pídele a Claude Code **todo el flujo** en un único prompt largo:

“Necesito un análisis completo de [archivo Excel entregado]: 1. Carga ambas hojas 2. Crea diccionario de variables 3. Estadística descriptiva (mean, sd, missing) 4. Estandarización de nombres y tipos 5. Variables nuevas: elite (rating  $\geq$  2200), years\_active 6. Visualizaciones: 3-5 gráficos clave 7. Resumen por país/región 8. Exporta reporte HTML completo”

**Nota:** Claude Code puede generar 300+ líneas de código pulido en **~1 minuto**.

---

## Rúbrica de Evaluación

| Criterio                          | Excelente                                                  | Bueno                                        | Aceptable            |
|-----------------------------------|------------------------------------------------------------|----------------------------------------------|----------------------|
| <b>Carga de datos</b>             | Ambas hojas cargadas, validadas                            | Cargadas correctamente                       | Cargadas con warning |
| <b>Diccionario de variables</b>   | Completo, tipos correctos, descripciones claras            | Completo, tipos ok                           | Incompleto           |
| <b>Estadística descriptiva</b>    | Completa (mean, sd, quantiles, missing)                    | Cubiertos principales                        | Solo summary()       |
| <b>Limpieza y estandarización</b> | Nombres normalizados, tipos correctos, sin inconsistencias | Nombres limpios, tipos mayormente ok         | Parcial              |
| <b>Variables nuevas</b>           | 2+ variables útiles, bien justificadas                     | 1 variable clara                             | Ninguna o trivial    |
| <b>Visualizaciones</b>            | 3+ gráficos informativos, leyendas claras                  | 2 gráficos ok                                | 1 gráfico o pobre    |
| <b>Documentación/Reporte</b>      | Reporte HTML/PDF claro, código comentado                   | Reporte básico, algunas notas                | Mínimo o ausente     |
| <b>Uso de IA</b>                  | Aprovecha Claude Code bien, itera claramente               | Usa IA para partes, código propio para otras | Mínimo uso de IA     |

---

## Entrega

Debes entregar:

1. **Script R** (actividad\_01\_limpieza.R o .qmd) con código comentado
2. **Dataset limpio** (archivo CSV con datos procesados)
3. **Reporte HTML o PDF** con:
  - Diccionario de variables (tabla formateada)
  - Resumen estadístico (por país, por edad, por rating)

- Mínimo 3 visualizaciones
  - Explicación de decisiones de limpieza
- 

## Ejemplo de Lo Que Genera Claude Code (Real)

Prompt enviado:

"Dataset de ajedrez, quiero age\_group (20-30, 30-40, 40-50, 50+) y contar jugadoras por grupo"

Código generado (segundos después):

```
data_clean <- data_clean %>%
  mutate(age_group = case_when(
    age >= 20 & age < 30 ~ "20-30",
    age >= 30 & age < 40 ~ "30-40",
    age >= 40 & age < 50 ~ "40-50",
    age >= 50 ~ "50+",
    TRUE ~ NA_character_
  ))

count_by_age <- data_clean %>%
  group_by(age_group) %>%
  summarise(n = n(), .groups = 'drop') %>%
  arrange(desc(n))

print(count_by_age)
```

**Sin IA:** Escribirías esto mismo en ~5 minutos. **Con IA:** 10 segundos, listo.

---

# Reflexión Final: IA como Herramienta, No Reemplazo

## Usa IA para:

- **Boilerplate:** cargas, librerías, transformaciones estándar
- **Exploración:** prueba múltiples enfoques rápidamente
- **Debugging:** “¿por qué falla este argumento?”
- **Documentación:** genera comentarios y explicaciones
- **Refactorización:** “limpia este código”

## NO uses IA para:

- **Decisiones financieras:** tú eliges qué variable crear, IA genera código
- **Validación:** verifica resultados siempre (no confíes ciegamente)
- **Comprensión:** entiende el código generado antes de usarlo
- **Pensar:** IA acelera la ejecución, no la estrategia

## La Mentalidad Correcta

“IA es como un copiloto competente: genera código, pero **TÚ** diriges el análisis.”

---

# Recursos Adicionales

[Claude Code Docs](#)

[Positron IDE](#)

[tidyverse en R](#)

[Mejores prácticas en Finanzas Cuantitativas](#)